



Università degli Studi di Salerno

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

MarcoJams: interazione tra MarcoSmiles e Google AI Magenta

Relatore

Prof. Rocco ZACCAGNINO

Candidato

Dario Trinchese

Matricola: 05121 07479

Anno Accademico 2021/2022

Indice

1	Abstract	1
2	Introduzione	2
3	Related Works	4
3.1	A.I. DUET	5
3.2	Theremin	6
3.3	Neural Arpeggiator	7
3.4	KAiKU music glove	7
3.5	Magenta Studio	8
3.6	Virtual Piano	9
3.7	GECO MIDI	10
4	Background	11
4.1	Il protocollo MIDI	11
4.1.1	Struttura e connessioni	11
4.1.2	Messaggi MIDI	13
4.1.3	Virtual MIDI Ports	15
4.2	Magenta	16
4.2.1	Magenta MIDI Interface	17
4.3	Unity3D	19
4.3.1	Assets, GameObjects e Scripting	19
4.4	MarcoSmiles	21
4.4.1	Training Scene	22
4.4.2	Play Scene	22
5	MarcoJams: interazione tra MarcoSmiles e Magenta	24
5.1	Tools	26
5.2	Setup	27
5.2.1	Conda	28
5.2.2	CUDA e cuDNN by NVIDIA	29
5.3	Configurazione dell'ambiente e installazione di Magenta	29
5.4	Interazione con Magenta	31
5.4.1	UMX-25 & Magenta	32

5.4.2	Errori riscontrati con l'uso delle porte virtuali	35
5.4.3	MarcoSmiles & Magenta	38
5.5	Modifiche all'interfaccia grafica di MarcoSmiles	39
5.5.1	GameObjects	40
5.5.2	Scripts	41
5.6	Esempio di interazione completa	46
6	Conclusioni	47
6.1	Considerazioni	47
6.2	Sviluppi futuri	48

Capitolo 1

Abstract

L'elaborato nasce dalle opportunità che la Computer Music, disciplina di intersezione tra musica ed informatica, mette a disposizione sul piano della creatività e dell'inventività. In letteratura, nel corso degli anni numerose sono state le idee e le creazioni dei cosiddetti "musimatici", studiosi delle due discipline. Il seguente lavoro, che gode delle opportunità creative che la Computer Music mette a disposizione, consiste nella progettazione e nello sviluppo di un metodo di interazione tra software di Computer Music già esistenti, *MarcoSmiles* e la *Magenta MIDI Interface*, con l'obiettivo di creare un nuovo elaborato originale, **MarcoJams**, frutto di una combinazione fantasiosa tra i due. MarcoJams è il prodotto della *sinergia* tra la *generazione melodica dal vivo* offerta da MarcoSmiles e le potenzialità dell'*intelligenza artificiale* della Magenta MIDI Interface; con MarcoJams la performance musicale generata dall'utente è rielaborata tramite modelli di reti neurali per la generazione di una risposta musicale dal vivo, creando un **interplay** tra utente e macchina. Verranno presentati gli strumenti e le tecnologie necessarie che hanno permesso l'ottenimento dell'interazione tra MarcoSmiles, software sviluppato nel 2019 dai ricercatori del Laboratorio di Computer Music dell'*Università degli Studi di Salerno*, e la Magenta MIDI Interface, progetto di ricerca frutto delle menti del *Google Brain Team*. Successivamente verrà definito un metodo semplice ed immediato per l'automatizzazione dell'interazione tra i software, cioè l'aggiunta e l'integrazione di un nuovo modulo all'interfaccia grafica di MarcoSmiles, così da ottenere una rivisitazione della classica modalità di utilizzo, estenderne le funzionalità e la potenza espressiva.

Capitolo 2

Introduzione

La Computer Music è una disciplina che si pone perfettamente nel punto mediano tra l'attività artistica (la musica) e l'attività scientifica (l'informatica), prelevando il meglio di entrambe le discipline. La sua nascita risale agli inizi degli anni 60', grazie ai grandi progressi dell'informatica e soprattutto grazie ai musicisti e alla loro innata curiosità di voler sperimentare la fusione della musica con le nuove tecnologie.

La finalità della Computer Music non è la sola generazione di nuove tracce musicali tramite strumenti informatici, né tanto meno il solo ed arido uso di uno strumento musicale collegato ad un dispositivo informatico, come si potrebbe pensare in un primo momento; essa consiste invece nella costante ricerca delle *affinità*, *corrispondenze* e *relazioni* che chiaramente esistono tra musica ed informatica. La ricerca di queste corrispondenze non è affatto vana: musica ed informatica sono *sorelle*, figlie della stessa madre, la *matematica*. Per Computer Music si intende quindi l'uso delle discipline informatiche nello studio e nella ricerca in ambito musicale ponendo in primo piano la **creatività**. La Computer Music non è associabile quindi alla semplice operazione di editing/montaggio di un file sonoro tramite computer od altri hardware, è qualcosa che si spinge concettualmente più avanti.

Il termine "Computer Music" fu coniato da *Max Mathews*, considerato il padre indiscusso della computer music. Sviluppò *MUSIC* nei laboratori della *Bell* [1] all'inizio degli anni 60, il primo di una lunga serie di software per la generazione di suoni tramite computer.

MarcoSmiles opera nell'ambito della Computer Music, propone un metodo alternativo e curioso per eseguire performance musicali, ponendo attenzione sull'uso del Leap Motion Controller, dispositivo di tracking dal costo ridotto. MarcoSmiles utilizza una rete neurale artificiale per permettere all'utente di suonare, tramite il *Leap Motion Controller* [2], uno strumento musicale virtuale totalmente personalizzabile.

Un altro grande esempio di applicazione degli strumenti dell'informatica nell'ambito della generazione artistica (non necessariamente solo la musica) è **Magenta** [3], un esperimento del Google Brain Team nato nel 2016. Il progetto Magenta è tutt'oggi in continuo sviluppo, mette a disposizione strumenti ed algoritmi per la generazione artistica su ogni piano, dalla musica alle arti figurative; nasce con l'intento di fornire agli artisti tutto ciò che di informatico necessitano per esprimere al meglio la loro creatività, stupire, affascinare.

Ponendo particolare attenzione sulla parola "creatività", questa tesi si pone il fine di mostrare ciò che di creativo è stato possibile *immaginare* e *realizzare* per MarcoSmiles, software nato con lo scopo di fornire, oltre ad uno strumento alternativo per la performance musicale, spazio vitale per la **fantasia** e l'**inventività** degli sviluppatori.

In particolar modo verrà descritto il modo con cui è stata ottenuta l'interazione tra *MarcoSmiles* e la *Magenta MIDI Interface* con l'obiettivo di rendere più accattivante e divertente la performance musicale, così come le nuove funzionalità introdotte.

Nel capitolo 2 verranno descritti alcuni lavori di natura affine a MarcoSmiles così come anche altri curiosi esperimenti di Computer Music. Nel capitolo 3 verrà fornito tutto il background necessario per entrare nel mondo del discorso. Nel capitolo 4 verranno descritti in dettaglio gli obiettivi prefissati e il modo con cui sono stati raggiunti, dalla progettazione allo sviluppo. Infine, nel capitolo 5 saranno descritti alcuni possibili spunti per lavori futuri, al fianco di alcune considerazioni finali.

Capitolo 3

Related Works

Nel seguente capitolo sono illustrati dispositivi e/o progetti software che potrebbero potenzialmente interessarci essenzialmente per due motivi: il primo motivo è che alcuni degli elementi illustrati sono dispositivi simili a MarcoSmiles, simili nel funzionamento, dispositivi che si appoggiano quindi sullo stesso concetto alla base di MarcoSmiles cioè utilizzare il movimento e la posizione delle mani e delle dita per generare suono o, più specificamente, segnali MIDI. Il secondo motivo è che altri elementi qui illustrati sono progetti software che si appoggiano sull'uso di Magenta, fulcro dell'intero lavoro descritto in questa tesi.

3.1 A.I. DUET

A.I. DUET [4] è un'applicazione web based sperimentale sviluppata da Google Creative Lab. Usa l'intelligenza artificiale per permettere all'utente di suonare ed attendere dall'applicazione una risposta alla melodia suonata in input, dando vita ad un simpatico duetto (questo schema è molto conosciuto in ambito musicale con il nome di "Botta e Risposta" [5]). L'applicazione è stata sviluppata usando TensorFlow, Tone.js e diversi tools della **Magenta MIDI Interface**.



Figura 3.1: Esempio di botta e risposta in musica

È molto semplice comprendere la modalità di interazione con l'applicazione. L'utente usa la tastiera (o il mouse o un dispositivo MIDI) per suonare un motivetto (i rettangoli *celesti* corrispondono alle note suonate dall'utente); l'applicazione elabora la chiamata dell'utente e, dopo poche battute, risponde generando sull'interfaccia grafica nuovi rettangoli (gialli) e suonando le note corrispondenti ai nuovi rettangoli. Durante l'interazione, l'interfaccia grafica fa fluttuare verso l'alto le note suonate, quindi una nota più in alto nell'interfaccia è stata suonata *prima* di una nota più in basso.



Figura 3.2: Esempio di utilizzo di A.I. DUET con tastiera MIDI

L'applicazione è disponibile sul web e pronta all'uso. Per interagire con l'applicazione è sufficiente usare la tastiera, il mouse premendo le note direttamente dall'interfaccia grafica dell'applicazione o ancora meglio usando una tastiera MIDI per una miglior esperienza di utilizzo. L'osservazione e lo studio della struttura di questa applicazione sperimentale è stato di *fondamentale importanza* per lo svolgimento dell'interno lavoro di tesi.

3.2 Theremin

Inventato nel 1919 dal fisico sovietico Sergeevič Termen, il *Theremin* [6] è il più antico strumento elettronico musicale. Si tratta di uno strumento musicale analogico, composto da una scatola (cabinet) originalmente di legno e due antenne situate sia sopra che al lato. All'interno della scatola è presente il vero e proprio nucleo elettronico dello strumento mentre le antenne servono per modulare tono e intensità del suono prodotto. Come la maggior parte degli strumenti elettronici dello scorso secolo, anche per il Theremin è stato previsto un nuovo modello perfezionato e dotato di interfaccia MIDI, il Moog Theremini [7].

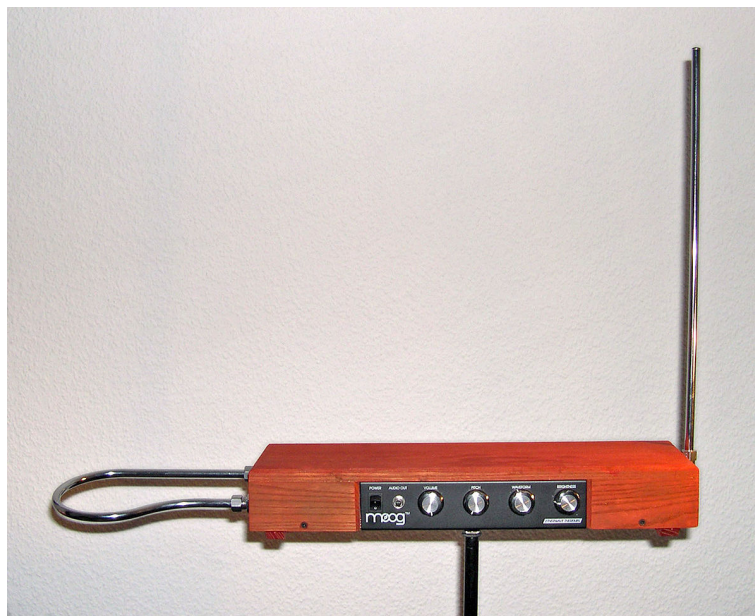


Figura 3.3: Theremin

3.3 Neural Arpeggiator

Neural Arpeggiator [8] è anch'essa un'applicazione web based, sviluppata da Tero Parviainen, membro della community **#MadeWithMagenta** [9]. Neural Arpeggiator è solo uno dei tantissimi esempi di applicazioni sviluppate dalla community e visitabili nella sezione Community Demos sul sito ufficiale di Magenta. Presenta un'interfaccia abbastanza simile ad A.I. DUET, sullo stesso stile. I comandi sono gli stessi di A.I. DUET, l'unico input richiesto è una *singola nota suonata costantemente*; l'applicazione genera su di essa un arpeggio casuale che l'utente sarà in grado di visualizzare ed ascoltare. Di questo arpeggio è possibile modificarne temperamento e durata. Neural Arpeggiator è stata sviluppata usando TensorFlow.js, Tone.js e Magenta.js con l'aiuto della rete neurale ImprovRNN preaddestrata dal team di Google Magenta.



Figura 3.4: Esempio di utilizzo di Neural Arpeggiator

3.4 KAIKU music glove

Il *KAIKU Music Glove* [10] è un controller MIDI indossabile, un singolo guanto sul quale sono posizionati dei sensori touch. Mentre una mano indossa questo speciale guanto, l'altra mano tocca i sensori presenti su di esso e questa interazione fa sì che il guanto generi segnali MIDI, inviati al PC via Bluetooth o via USB. Questo dispositivo è generalmente utilizzato per la didattica della musica più che per obiettivi di performance; il produttore, Taction Enterprises Inc., lo ha strutturato in modo tale che fosse estremamente semplice da usare per insegnare le note e gli intervalli in modo efficiente,

sia per l'insegnante che per l'alunno. Colui che intende utilizzare lo strumento deve necessariamente assegnare ad ogni sensore sul guanto la nota o il messaggio MIDI da inviare.



Figura 3.5: KAIKU Music Gloves

3.5 Magenta Studio

Magenta Studio [11] è una collezione di plugin costruiti sugli strumenti e modelli open source di Magenta. Usano tecniche di machine learning all'avanguardia per la music generation. Gli strumenti che compongono Magenta Studio sono disponibili sia come applicazioni stand-alone sia plugins per *Ableton Live* [12], software DAW (Digital Audio Workstation) molto noto nel mondo della produzione musicale.

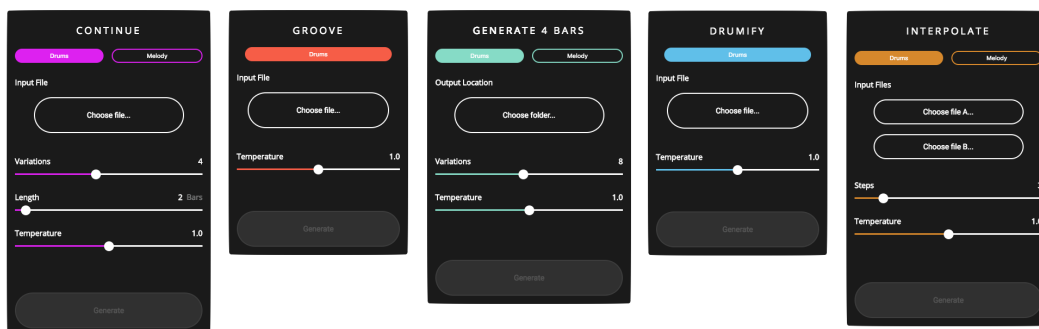


Figura 3.6: I plugin di Magenta Studio

Magenta Studio contiene cinque plugin in tutto, ecco una breve descrizione per ognuno di essi:

- **Continue:** usa la potenza predittiva delle reti neurali ricorrenti per generare note che possono potenzialmente seguire il beat o la melodia data in input. Dato un

file in input, può estenderlo fino a 32 battute

- **Groove:** prende in input un drum pattern e per esso smussa parametri come tempismo e velocità d'esecuzione in modo da ottenere lo stesso "feel" della performance di un batterista reale, *umanizzando il suono*
- **Generate:** simile al Continue, non richiede input. Groove non *continua* melodie bensì le genera da zero. Dati in input parametri come il temperamento e il numero di variazioni, genera un file MIDI di 4 battute nella directory di destinazione
- **Drumify:** crea accompagnamenti di batteria basati sul ritmo dell'input. Molto utile per accompagnare linee di basso o generare nuove idee a partire da un ritmo grezzo e tamburellato.
- **Interpolate:** Diversamente dagli altri plugin, Interpolate prende in input **due** melodie (o beats) per essere genera fino a 16 risultati che combinano le qualità dei due input. È estremamente utile per unire idee o generare un modo dolce e piacevole per passare da una melodia ad un'altra.

3.6 Virtual Piano

Virtual Piano [13] è un applicazione che fa uso del Leap Motion Controller per permettere all'utente di suonare un pianoforte virtuale. L'interfaccia grafica presenta un pianoforte tridimensionale e lo schema stilizzato delle mani dell'utente, mostrato sulla base dei dati prelevati in tempo reale dal Leap Motion Controller. Grazie a questa applicazione sviluppata nel 2017 e consultabile nella Leap Motion Gallery ufficiale il Leap Motion Controller trova spazio di utilizzo anche nella Computer Music.

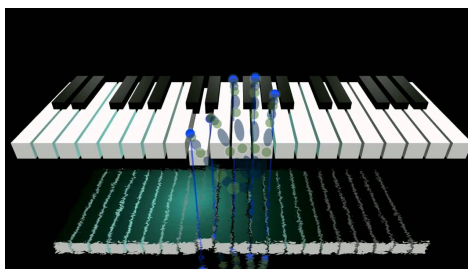


Figura 3.7: Virtual Piano

3.7 GECO MIDI

Presente anch'essa nella Leap Motion Gallery, *GECO MIDI* [14] è un'applicazione che permette di trasformare il proprio Leap Motion Controller in un vero e proprio controller MIDI, in modo da poter inviare note, modulare pitchband, channel pressure, utilizzare comandi rapidi, tutto tramite l'utilizzo di gesture che l'applicazione è in grado di riconoscere. L'interfaccia è totalmente personalizzabile, è in grado di collegarsi a qualsiasi porta di output MIDI, come ad esempio sintetizzatori hardware, sia su Windows che su MacOS.



Figura 3.8: GECO MIDI in azione

Capitolo 4

Background

4.1 Il protocollo MIDI

Il protocollo **MIDI** [15] (*Musical Instrument Digital Interface*) nasce all'inizio degli anni 80 grazie a Dave Smith e Chat Wood di Sequential Circuits [16]. All'epoca non ancora esisteva uno standard di comunicazione tra strumenti musicali elettronici; protocolli di comunicazione erano già stati ideati da alcune case produttrici come Roland e Oberheim ma si trattavano di protocolli il cui utilizzo portava alla generazione di soluzioni complesse per l'interazione tra un numero discreto di strumenti. Nel 1981 furono proposte le prime specifiche del MIDI in un documento pubblicato sotto il nome di "The complete SCI MIDI". Grazie al protocollo MIDI è garantito il funzionamento tra dispositivi di costruttori diversi, si ha a disposizione un protocollo semplice da programmare e diffuso in tutto il mondo, modificato e perfezionato nel tempo.

4.1.1 Struttura e connessioni

Il protocollo MIDI definisce un insieme di regole che permettono di tradurre le azioni del musicista eseguite su uno strumento musicale (il dispositivo **master**) in una sequenza di 0 ed 1 (a loro volta tradotti in segnali discreti compresi tra 0 e +5 volts). Questa sequenza prende il nome di messaggio MIDI, è grande uno o più byte e possono indicare le più svariate azioni che il musicista esegue sullo strumento, come "Suonare la nota C5" oppure "Aumenta la velocità del metronomo" o ancora "Diminuisci il pitch". Un messaggio MIDI viene ricevuto da un dispositivo (il dispositivo **slave**) che saprà quindi "cosa farsene". Un esempio di coppia master-slave può essere un controller ed un sintetizzatore: il controller invia il messaggio MIDI "Suona la nota A4" e il

sintetizzatore, ricevendo il messaggio, suona la frequenza corrispondente. Un messaggio MIDI quindi *non* è un formato audio né contiene alcun suono, ma è una sequenza di bit che traduce le azioni del musicista. Affinché una coppia di dispositivi MIDI possa comunicare è necessario che esse siano dotati di un'adeguata interfaccia hardware, detta interfaccia MIDI. L'interfaccia è composta da tre tipologie di connessione:

- **MIDI IN** porta a cui arrivano i dati. Questa porta deve essere sempre collegata con una porta MIDI Out oppure Thru.;
- **MIDI OUT** porta da cui escono i dati e vanno verso un MIDI In.;
- **MIDI THRU** che consiste in una copia esatta dei dati che arrivano al MIDI In, serve per collegare più dispositivi allo stesso master.

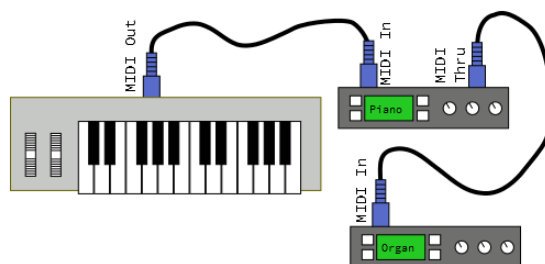


Figura 4.1: Daisy Chain, esempio classico di connessione MIDI

È bene sapere che, sebbene la maggior parte dei dispositivi MIDI recenti utilizzi l'USB (Universal Serial Bus) come interfaccia hardware per l'invio e la ricezione dei dati, essa non è l'interfaccia hardware originalmente adottata per la trasmissione dei messaggi MIDI. Infatti è possibile osservare come, nei dispositivi meno recenti, l'interfaccia MIDI sia composta da ben tre porte, rispettivamente per il Midi In, Midi Out e Midi Thru. A seconda della sua natura e scopo di utilizzo, uno strumento elettronico MIDI Legacy poteva non disporre di tutte le porte.



Figura 4.2: MIDI Legacy Ports

4.1.2 Messaggi MIDI

Un messaggio MIDI è costituito da diversi byte, divisi in due tipologie: Status Byte e Data Byte. Un messaggio MIDI è sempre composto da uno status byte all'inizio, seguito da uno o due data bytes.

- lo **status byte** definisce sia la tipologia del comando MIDI (i 3 bit più significativi dopo il primo) sia il canale di riferimento (i restanti 4 bit meno significativi). Essendo 4 bit dedicati alla specifica del canale su cui viaggerà il comando MIDI, è possibile indirizzare comandi MIDI su $2^4=16$ canali. Stesso discorso vale per i 3 bit dedicati alla tipologia di comando, rendendo possibile la generazione di $2^3=8$ comandi diversi. Lo status byte può essere visto come la *testa* del messaggio MIDI.
- il **data byte** rappresenta invece il valore richiesto per il parametro specificato nello status byte, può essere visto come il *payload* del messaggio MIDI.

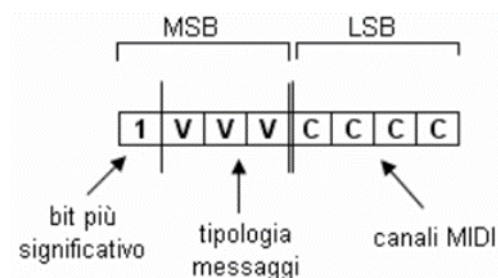


Figura 4.3: Struttura di uno *Status Byte* MIDI [15]

Effettivamente sia status byte che data byte sono una sequenza di otto cifre binarie. Come fa quindi il dispositivo che riceve questa sequenza a *distinguere status bytes dai data bytes*? Grazie al bit più significativo! Esso ha infatti valore **1 per gli status byte** e valore **0 per i data bytes**. Quando un dispositivo riceve un byte con MSB (Most Significant Bit) pari ad 1, sa che i successivi 3 bit indicando la tipologia del messaggio e gli ultimi 4 bit indicano il canale; quando invece esso riceve un byte con MSB pari a 0, sa che i restanti 7 bit indicano un valore (da 0 a 127) che fa necessariamente riferimento ad uno status byte ricevuto in precedenza.

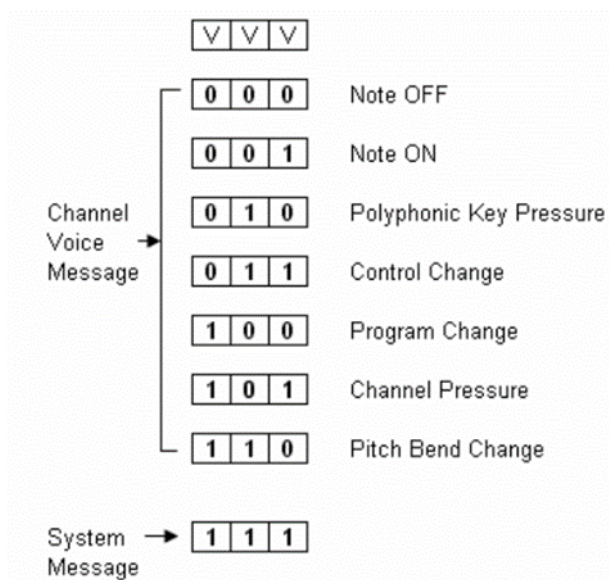


Figura 4.4: bit per la tipologia di messaggi

I messaggi MIDI si dividono poi in due categorie:

- **Channel Message**, fanno riferimento ad uno solo dei 16 canali MIDI disponibili, divisi a loro volta in *Voice* (indicano le vere e proprie note) e *Mode* (indicano come gestire i messaggi Voice, ad esempio Pitch o Bend)
- **System Message**, fanno riferimento all'intera network musicale, possono essere di tipo *System Common*, *System Real Time* oppure *System Exclusive*

4.1.3 Virtual MIDI Ports

Network musicali reali sono costituiti da strumenti elettronici hardware che interagiscono tra di loro inviandosi segnali MIDI sulle loro interfacce fisiche (che possono consistere in una porta USB oppure nelle porte MIDI Legacy). Per poter creare un network musicale virtuale, cioè **emulato all'interno di un personal computer**, è strettamente necessario creare porte MIDI virtuali ed interagire con esse. Una porta virtuale MIDI può essere usata per inviare e ricevere messaggi MIDI da/verso applicazioni presenti sulla stessa macchina.

Per poter giustificare l'esistenza delle porte MIDI virtuali basta pensare ad un esempio di network musicale virtuale dove è presente un applicativo in grado di generare segnali MIDI (come ad esempio un controller virtuale) ed un software **DAW** (come *Ableton Live* [12]) dove è attivo un sintetizzatore virtuale; nessuno di questi due componenti è fisico, entrambi coesistono virtualmente sulla stessa macchina. Per permettere l'interazione tra di essi è necessario definire per entrambi un interfaccia virtuale MIDI, dove il *Midi Out Virtuale del controller* è collegato al *Midi In Virtuale del sintetizzatore*.

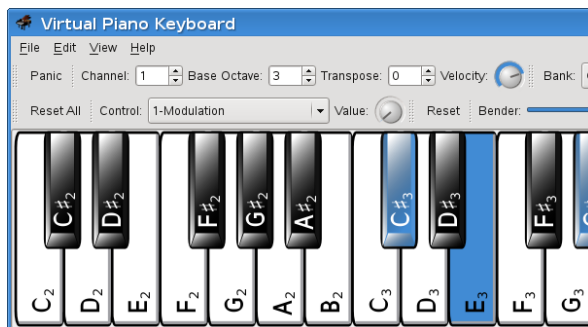


Figura 4.5: VMPK: Esempio di controller MIDI virtuale

Esistono diversi applicativi che permettono la creazione e gestione di porte virtuali MIDI; il software per Windows più comunemente utilizzato per questo scopo è **loopMIDI** [17] by Tobias Erichsen.

4.2 Magenta

Magenta [3] è un progetto di ricerca open source, consultabile su GitHub [18], che intende **esplorare il ruolo del Machine Learning** come uno strumento utile nei **processi creativi**. È nato dalle menti dei membri del **Google Brain Team** nel 2016 e nel corso degli anni ha ricevuto tanti altri contributi significanti anche fuori dal team di progetto. Magenta nasce dalla domanda *"Possiamo usare il Deep Learning e il Reinforcement Learning per generare qualcosa di creativo?"*. Si basa su *Tensorflow* [19], anch'esso prodotto delle menti del Google Brain Team, una libreria software open-source per il machine learning che fornisce moduli sperimentati e ottimizzati per la realizzazione di algoritmi di diversi tipi, implementabile su diverse piattaforme e alla base di diversi prodotti commerciali Google come il **riconoscimento vocale**, **Gmail**, **Google Foto**.



Figura 4.6: Logo ufficiale di Magenta

Con il progetto Magenta gli sviluppatori del Google Brain Team sono stati in grado di sviluppare algoritmi di deep learning e reinforcement learning per la generazione di **musica**, **immagini**, **disegni** e tante altre forme di espressione artistica. Come dichiarato da uno dei ricercatori del Google Brain Team, i primi algoritmi ideati in Magenta furono appunto quelli per la *song generation* [20], ad essi gli sviluppatori si mossero verso la creazione di algoritmi che gestissero video e altre forme artistiche.

4.2.1 Magenta MIDI Interface

Il 2 agosto 2016 Google Brain Team annuncia la Magenta MIDI Interface, un'interfaccia tra musicisti e Tensorflow. Grazie a questa interfaccia di semplice utilizzo è possibile permettere a *controller e sintetizzatori* (fisici o virtuali che siano) *di interagire con un modello Tensorflow*.

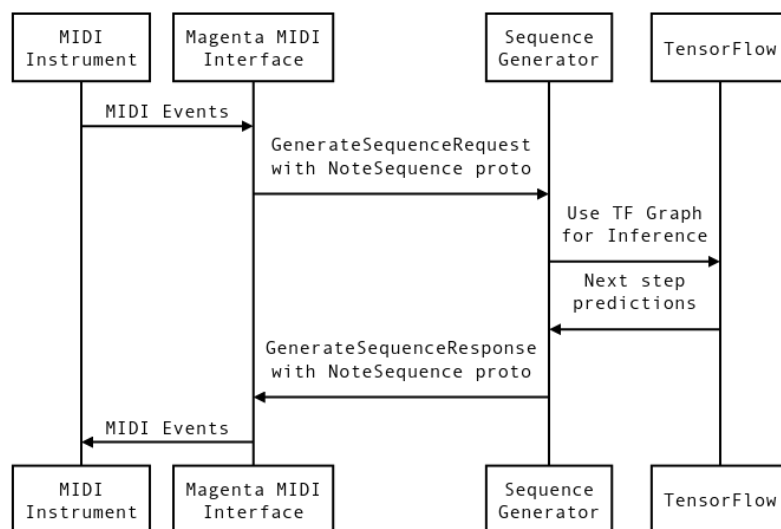


Figura 4.7: Sequence Diagram della Magenta MIDI Interface

L'interazione con la Magenta MIDI Interface è strutturata su **quattro livelli**. Il livello più superficiale è quello degli strumenti MIDI, che possono essere fisici o virtuali. Lo strumento MIDI master invia eventi MIDI alla Magenta MIDI Interface. La Magenta MIDI Interface inoltra una richiesta di generazione della risposta al Sequence Generator, un modulo software di machine learning incaricato di *ricevere in input la sequenza* generata dal dispositivo collegato all'interfaccia e dare in output alla Magenta MIDI Interface *la risposta generata sulla base del modello scelto*, grazie all'uso dei moduli offerti da Tensorflow. La risposta è inoltrata allo strumento MIDI slave, sul quale è possibile osservare e/o ascoltare la risposta.

Insieme all'interfaccia, Google Brain Team ha rilasciato un gran numero di **modelli Tensorflow** da loro **preaddestrati** e un **API** che permette agli sviluppatori di collegare facilmente i propri modelli alla MIDI Interface, disponibili su GitHub [21]

Dalla pubblicazione della Magenta MIDI Interface nel 2016 tantissimi sono stati i contributi della community **#MadeWithMagenta** [9] nello sviluppo di applicazioni che si basassero su tale interfaccia. Gli esempi discussi nel capitolo 2, *A.I. DUET* e *Neural Arpeggiator* sono soltanto due tra gli innumerevoli progetti frutto della community, consultabili sul sito ufficiale di Magenta. Ognuno di questi progetti usa direttamente o indirettamente la Magenta MIDI Interface nei modi più creativi possibili.

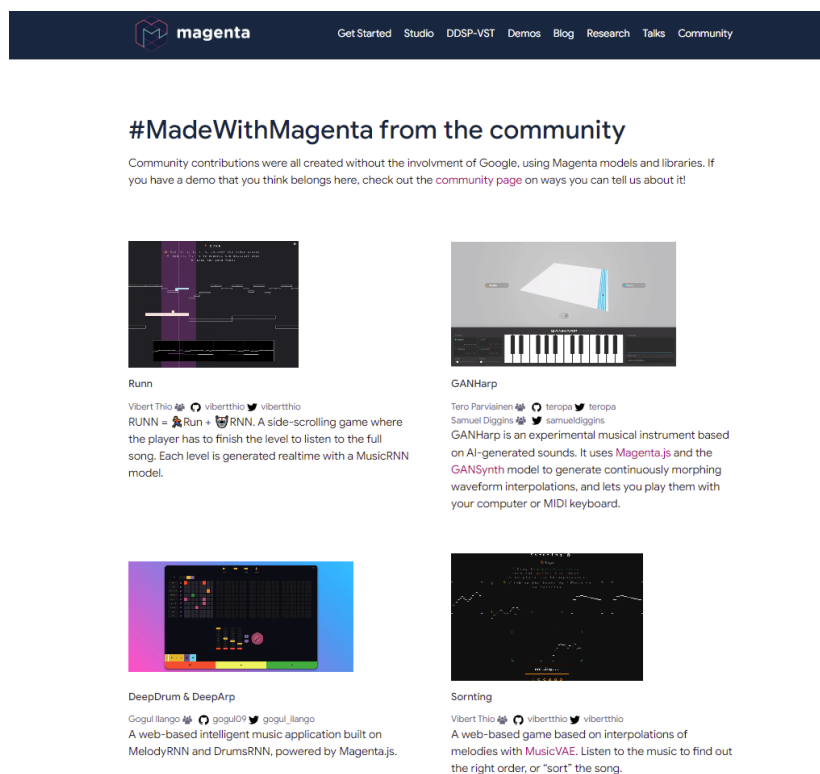


Figura 4.8: Progetti della community sul sito ufficiale di Magenta

Infatti è bene specificare tra gli obiettivi del progetto Magenta, oltre che chiaramente sviluppare algoritmi che permettono di fondere il Deep Learning con la generazione di prodotti artistici, c'è l'obiettivo implicito di *distinguere il ruolo degli sviluppatori da quello degli artisti* "Fornire agli artisti gli **strumenti** tali da riuscire a raggiungere da qui a qualche anno idee in grado di stupire il pubblico, idee che *neanche gli sviluppatori di Magenta sarebbero stati in grado di raggiungere.*" - **Douglas Eck**, ricercatore presso Google, durante la presentazione di Magenta al **Google I/O '17** [22]

4.3 Unity3D

Unity3D [23] è un game development engine che permette di creare videogame (o non necessariamente videogame, anche semplici applicazioni interattive) ed esportarli su più piattaforme (Windows, Linux, Mac) e più dispositivi (iOS, Android). I prodotti possono essere applicazioni sia tridimensionale che bidimensionali, dispone degli strumenti per lo sviluppo di applicazioni che si prestano all'uso di sensori e di visori per la realtà aumentata, è **gratuito** e per questo la maggior parte degli sviluppatori Unity decide di sviluppare il proprio prodotto con questo engine. Mette a disposizione un ambiente di sviluppo completo, la logica applicativa può essere sviluppata in *C#*, *JavaScript* oppure *Boo*. L'interfaccia di Unity3D per gli sviluppatori si compone di pannelli, che possono essere personalizzati e ridimensionati dall'utente a seconda delle sue esigenze durante lo sviluppo.

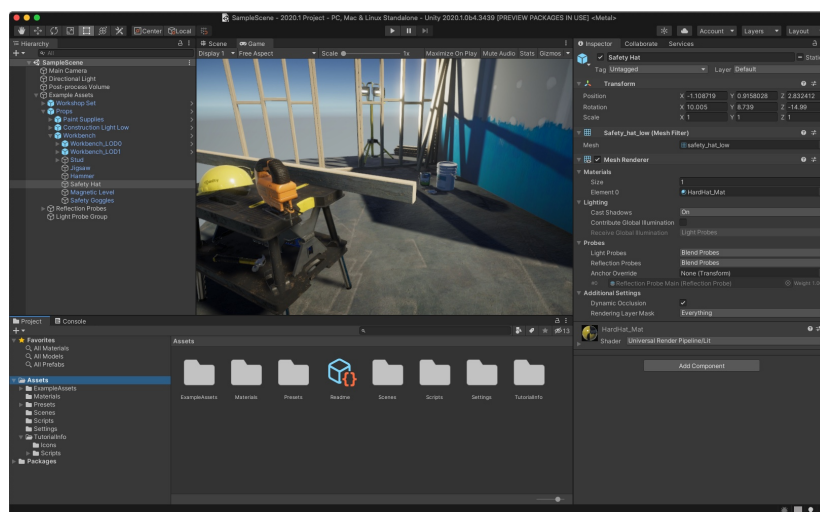


Figura 4.9: Interfaccia di Unity3D

4.3.1 Assets, GameObjects e Scripting

Un **Asset** è un qualsiasi elemento che è possibile usare nel proprio gioco o progetto. File multimediali come foto (JPG, PNG, TIFF), video, audio (WAV, MP3, OGG) sono assets, così come anche foto, modelli 3D, animazioni, textures, eccetera. Unity3D così come concepito non è un editor o generatore di assets bensì un software che permette la combinazione di assets tramite logica applicativa con lo scopo di creare un applicazione interattiva o videogioco. Sebbene esso mette a disposizione **comandi per**

creare primitive 3D come parallelepipedi o sfere, il luogo adatto per creare modelli 3D (necessario nel caso di modelli 3D più complessi) sono software come *Blender* o *3D Studio Max*. È possibile poi importare il lavoro di modellazione svolto su questi software come oggetto tridimensionale all'interno del progetto Unity3D.

Un **GameObject** è un qualsiasi oggetto correntemente presente ed attivo nella scena. Ad ogni GameObject è assegnato una lista di Components che ne indicano le più svariate proprietà. Ogni GameObject possiede necessariamente il component Transform, che ne indica la posizione all'interno dello spazio della scena. Altri components assegnabili ad un GameObject possono essere Rigidbody (per indicare che tale oggetto deve rispondere alle leggi della fisica), Physic Material (per associare all'oggetto delle proprietà fisiche come la rugosità, l'attrito e il rimbalzo), eccetera. È possibile accedere alla lista dei components di un GameObject sia tramite l'interfaccia (pannello Inspector) sia programmaticamente tramite script.

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class Rotate : MonoBehaviour
6  {
7      // Start is called before the first frame update
8      void Start()
9      {
10         ...
11     }
12
13     // Update is called once per frame
14     void Update()
15     {
16         ...
17     }
18 }
```

Figura 4.10: Come si presenta uno script di default

Uno **Script** descrive il comportamento di un elemento che costituisce la scena, che sia un GameObject o la Camera o le proprietà della scena, tutto ciò che può essere modificato tramite accesso programmatico. Lo scripting in Unity si basa sullo standard MonoBehaviour. Ogni script in Unity deriva di default dalla classe MonoBehaviour. Questo standard permette di *collegare i propri script ai GameObject* e di *intercettare eventi* come **Start** (il momento in cui l'oggetto inizia ad esistere nella scena), **Update**

(aggiornamento del frame), **FixedUpdate** (timestep fisico), **onCollisionEnter** e **onTriggerEvent** (quando si verificano collisioni) e **onDestroy** (quando l'oggetto viene distrutto, scomparendo dalla scena).

4.4 MarcoSmiles

MarcoSmiles è un software per la Human Interaction within Computer Music sviluppato nel 2019 da tesisti e membri del laboratorio di Computer Music presso l'Università degli Studi di Salerno. L'idea di MarcoSmiles nasce da un ex-sassofonista di nome "Marco" non più in grado di suonare per causa di [inserire malattia] che lo impedisce di reggere il peso di un sassofono. Il concetto alla base di MarcoSmiles è quello di generare note usando un **Leap Motion Controller** [2], un modulo ottico per il tracciamento della posizione delle mani, sulla base di modelli neurali addestrati dall'utente nella stessa applicazione. La versione attuale di MarcoSmiles è la **3.0** sviluppata nel 2021 da *Daniele Salerno*, anch'esso tesista presso il laboratorio di Computer Music. Con la versione 3.0 sono stati apportati notevoli miglioramenti all'interfaccia grafica e alla User Experience in generale; inoltre lo strumento è stato trasformato in un *vero e proprio dispositivo MIDI virtuale*. La versione 2.0 limitava la performance all'ascolto delle note sul sintetizzatore locale all'applicazione; grazie alle modifiche apportate da Daniele Salerno attualmente MarcoSmiles è inoltre in grado di generare segnali MIDI su un porta MIDI virtuale. Specificare che MarcoSmiles sia ad oggi uno strumento MIDI virtuale è di *fondamentale importanza* e questo sarà giustificato successivamente nel Capitolo 4.



Figura 4.11: Main Scene

Dal punto di vista dell'interfaccia, MarcoSmiles si compone (oltre alla Main Scene) di due sezioni. Segue una loro breve descrizione

4.4.1 Training Scene

In questa scena è possibile *addestrare la rete neurale* creando un nuovo dataset o modificandone uno già esistente. Per insegnare alla rete neurale una nuova posizione per una nota è sufficiente selezionare la nota dal pianoforte nella scena, tenere ferme le dita nella posizione scelta (assicurandosi che il Leap Motion abbia riconosciuto le mani), premere il tasto verde e attendere il countdown. Il tasto rosso serve per eliminare l'ultima configurazione acquisita. Il pulsante con l'icona della semiminima serve per insegnare al sistema come riconoscere la "pausa" musicale.

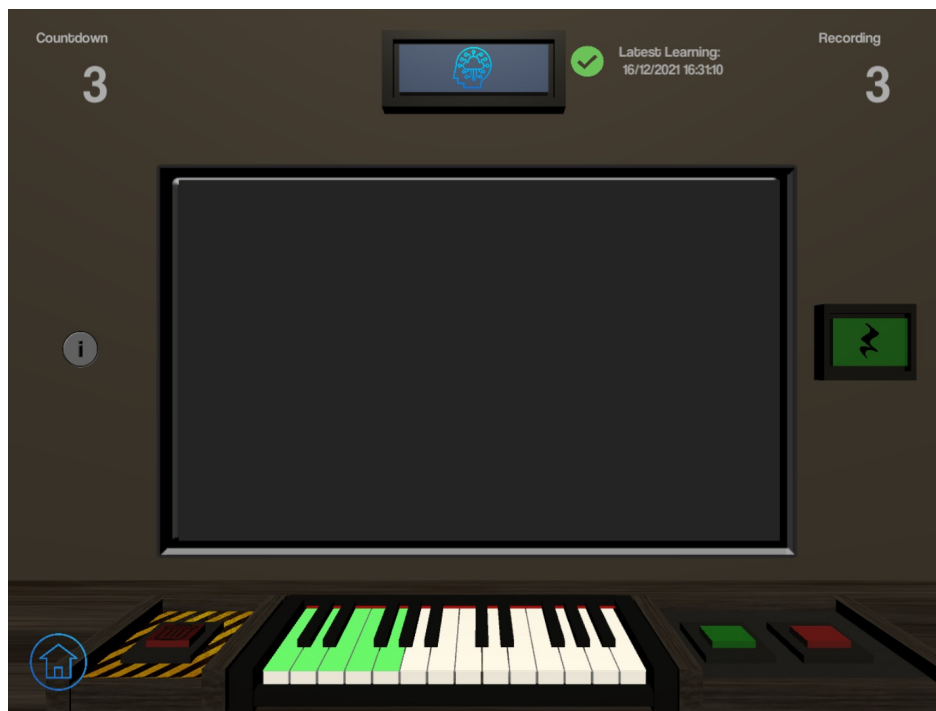


Figura 4.12: Learn Scene

4.4.2 Play Scene

Una volta creato un dataset basterà selezionarlo nella Main Scene e premere sul pulsante Play per trovarsi nella Play Scene dove si può dare inizio alla performance musicale. MarcoSmiles acquisisce in real-time i dati relativi alla posizione delle dita, la rete neurale analizza i dati e risponde, *se il tasto MIDI ON è spento*, riproducendo dal

sintetizzatore locale la nota riconosciuta secondo ciò che è dettato dal dataset selezionato. *Se il tasto MIDI ON è attivo*, nessun suono verrà riprodotto bensì verrà generato un **evento MIDI** sulla **porta virtuale MarcoSmiles**, aperta e gestita dal software *loopMIDI* [17].



Figura 4.13: Play Scene

Capitolo 5

MarcoJams: interazione tra MarcoSmiles e Magenta

Prima di partire con la descrizione dettagliata delle modifiche apportate a MarcoSmiles bisogna **delineare l'obiettivo**, cioè il verso di marcia dell'intero lavoro. Sicuramente, l'obiettivo di tutti coloro che hanno fatto o faranno parte del team degli sviluppatori di MarcoSmiles è quello di *apportare modifiche e miglioramenti* al software nella direzione della *Human Interaction, usabilità ed interazione con sistemi già esistenti*.

L'obiettivo, per questa volta, è quello di ottenere un certo grado di **interazione tra MarcoSmiles e Magenta** così da ottenere un'esperienza d'uso sulla falsa riga di A.I. DUET descritto nel Capitolo 2; disporre MarcoSmiles di un modulo che permetta di avviare un duetto con la Magenta MIDI Interface, che leggerà in input i messaggi MIDI generati da MarcoSmiles e darà in output una sua response [5] generata.

In che modo vogliamo ottenere questo? L'idea generale alla base del lavoro qui descritto si appoggia sull'ipotesi che MarcoSmiles sia, grazie alle modifiche apportate da Daniele Salerno nel suo lavoro di tesi, *un dispositivo MIDI a tutti gli effetti*, che genera messaggi MIDI su una porta virtuale di nome "MarcoSmiles". La Magenta MIDI Interface [Figura 4.7], dualmente, si compone di una struttura a più livelli dove il primo livello è quello degli strumenti MIDI. Questo suggerisce che un modo per ottenere l'interazione tra MarcoSmiles e Magenta può esistere!

Andando più specificamente nei particolari e riosservando la figura 4.7 si può osservare come il livello superficiale della Magenta MIDI Interface è il livello degli strumenti MIDI. Magenta **non limita** l'utilizzo dell'interfaccia ai soli dispositivi fisici, pertanto MarcoSmiles è uno strumento MIDI candidato per l'interazione; si potrebbe pensare di avviare la Magenta MIDI Interface specificando come porte di input ed output quelle di MarcoSmiles.

In questo modo i segnali MIDI generati da MarcoSmiles entrerebbero in input all'interfaccia, verrebbero elaborati dal SequenceGenerator che, dopo l'interazione con Tensorflow sulla base del modello scelto, risponderebbe con una nuova sequenza di segnali MIDI canalizzabili verso una qualsiasi porta MIDI di entrata; da questa porta saremmo in grado di osservare il *frutto della creatività artificiale* dell'interfaccia.

Riassumendo con uno schema, avremmo davanti uno scenario del genere:

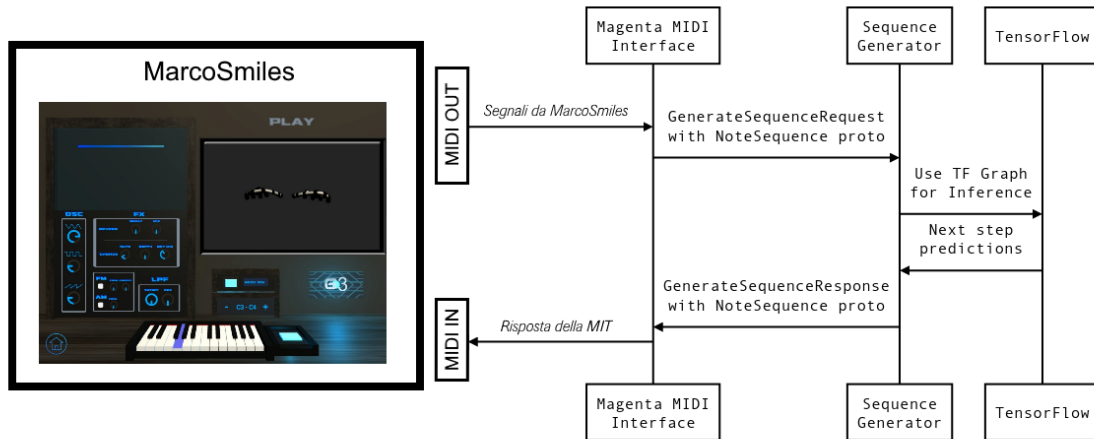


Figura 5.1: Idea di interazione tra MarcoSmiles e MIT

5.1 Tools

Per permettere il funzionamento di MarcoSmiles, l'interazione con Magenta, fare troubleshooting e debugging è stato necessario l'utilizzo di questi tools:

- **loopMIDI** [17], già introdotto nel capitolo 3.1.3, software sviluppato da Tobias Erichsen per la gestione delle porte MIDI virtuali. Grazie all'interfaccia minimale è possibile osservare quali sono le porte correntemente attive/mutate e per ogni porta osservare la quantità di dati che l'hanno attraversata e il throughput corrente.

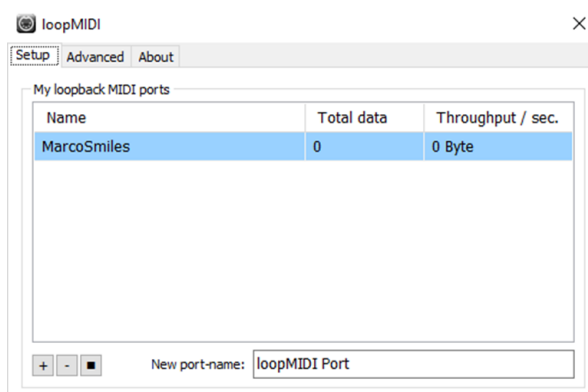


Figura 5.2: loopMIDI

- **Midi View** [24], sviluppato da Haute Technique B.V., è un'applicazione che permette di monitorare con buon dettaglio il flusso di dati che attraversa le porte MIDI correntemente attive sul dispositivo, che siano fisiche o virtuali. È stato estremamente utile per alcune operazioni di troubleshooting.

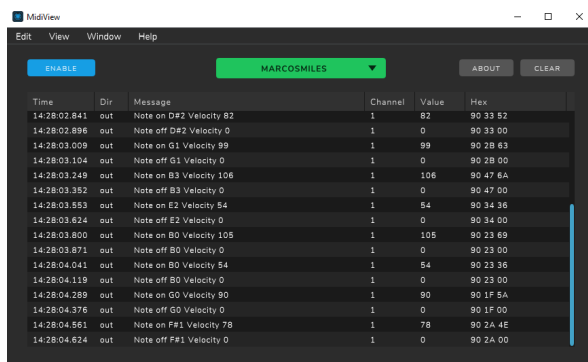


Figura 5.3: Midi View

- **MIDI Tools** [25], sviluppato da Mountain Utilities, è un *coltellino svizzero* per il MIDI, monitora il flusso MIDI sulle porte attive, fornisce strumenti per filtrare messaggi, creare file di log, mette a disposizione una tastiera MIDI virtuale totalmente personalizzabile in grado di generare segnali MIDI su una porta selezionata.

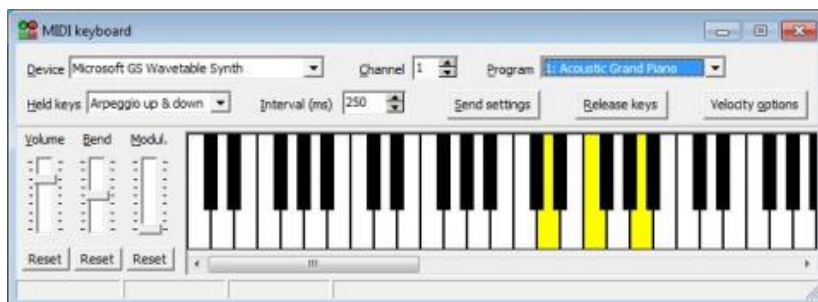


Figura 5.4: MIDI Tools

5.2 Setup

Per poter usare gli strumenti di Magenta, come la *Magenta MIDI Interface*, è necessario installare il **pip package Magenta**. L'installazione di tale package non è un'operazione immediata dal momento che esso dipende da una moltitudine di componenti software e frameworks.

È bene evidenziare che tali dipendenze sono state scoperte, durante la fase di studio, con un approccio naturalmente **top-down**; ogni errore incontrato in una certa componente software ha suggerito l'esistenza di una dipendenza verso un'altra componente software e così via, fino all'ultima componente, cioè quella indipendente e dall'installazione immediata.

Nella restante parte del Capitolo 4 si intende descrivere i passaggi (su Windows) per ottenere la corretta installazione del Magenta PIP Package con un approccio **bottom-up**, partendo prima dai componenti indipendenti fino a raggiungere all'*installazione corretta del pip package Magenta*, grazie al quale si potrà poi usare la Magenta MIDI Interface, così come molti altri strumenti sempre presenti nello stesso pip package.

5.2.1 Conda

Conda [26] è un sistema open source in grado di gestire packages ed environments per molti linguaggi. È stato originariamente creato per Python. Grazie a Conda è semplice creare ambienti di sviluppo sul proprio PC, salvare, caricare e switchare tra ambienti già presenti. Conda servirà a creare un **environment indipendente** dall'installazione di Python presente sulla propria macchina. Il pip package Magenta, così come tutti gli altri packages dai quale dipende, saranno installati in questo environment. Per installare Conda è sufficiente seguire la *guida ufficiale* [27]. In seguito all'installazione il comando "conda" non verrà riconosciuto nel terminale. Per permettere al terminale di riconoscere il comando "conda" è necessario modificare la variabile d'ambiente PATH aggiungendo tra i suoi record i percorsi di installazione di Conda.

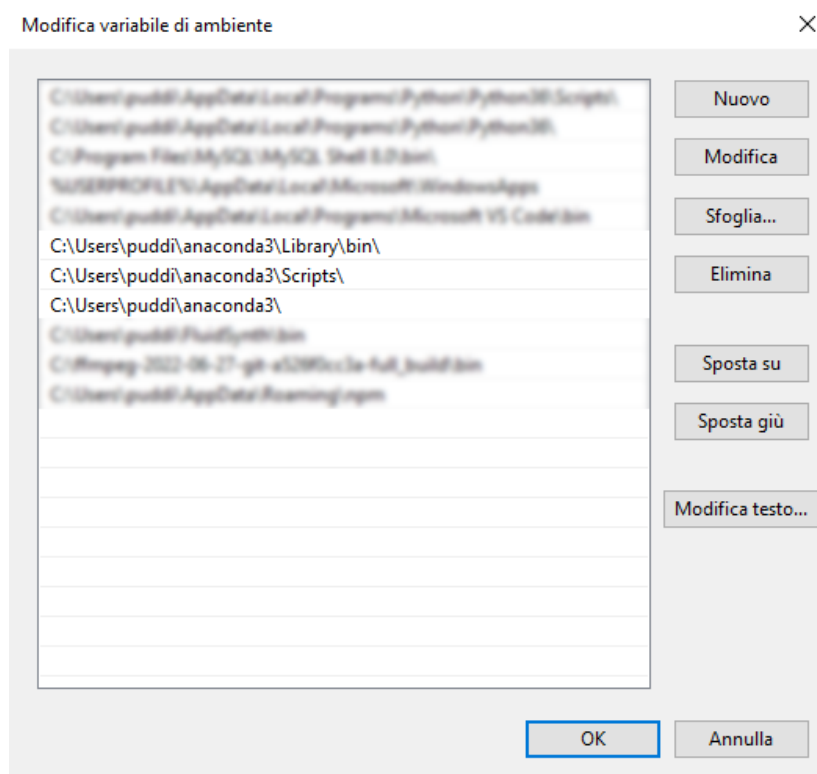


Figura 5.5: Nuovi record per Conda nella variabile d'ambiente PATH

Basterà riavviare il terminale e rieseguire il comando "conda" per accorgersi che il comando è riconosciuto. Con questo, Conda è correttamente installato.

5.2.2 CUDA e cuDNN by NVIDIA

NVIDIA cuDNN (NVIDIA CUDA Deep Neural Network library) [28] è una libreria, basata su *NVIDIA CUDA* [29], che mette a disposizione primitive per le Deep Neural Networks. Sarà necessario installare ed utilizzare questa libreria per poter sfruttare al meglio la potenza delle GPU. Sarà inizialmente necessario scaricare ed installare NVIDIA CUDA Toolkit 10.0 direttamente dal sito ufficiale [30]. Successivamente va installato NVIDIA cuDNN e fare riferimento ai percorsi d'installazione nei record della variabile di sistema PATH (non d'ambiente), proprio come per Conda.

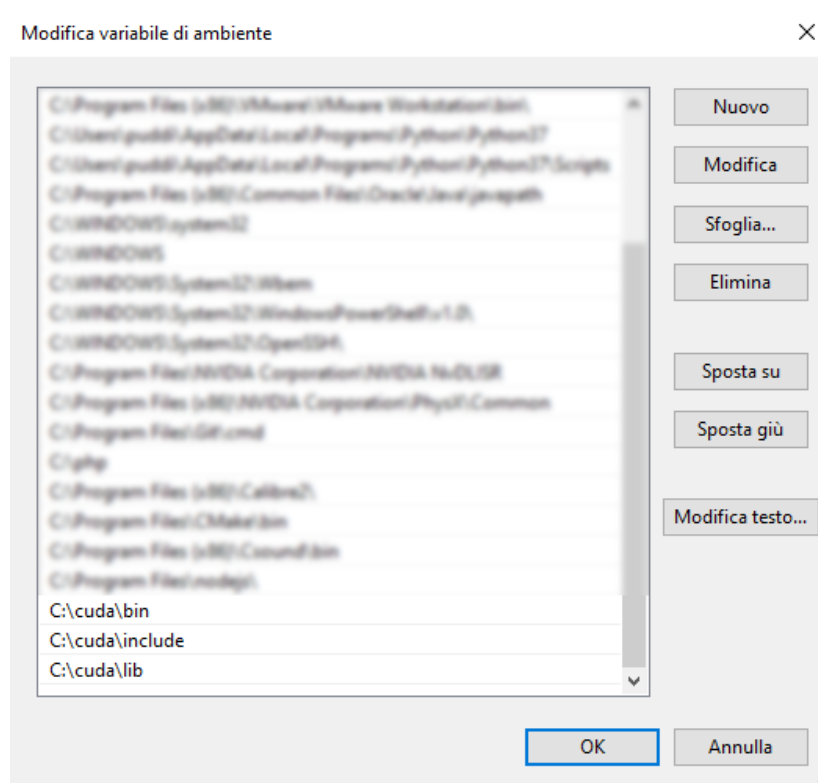


Figura 5.6: Nuovi record per CUDA nella variabile di sistema PATH

5.3 Configurazione dell'ambiente e installazione di Magenta

Ora che sia Conda che cuDNN sono stati installati, il prossimo passaggio è quello di creare un ambiente virtuale con Conda e configurarlo in modo tale da permettere l'installazione del pip package Magenta. Prima di questo va installato *Microsoft Visual C++ Redistributable*, è possibile installarlo (o aggiornarlo, se già dovesse essere presente sulla macchina) usando Microsoft Visual Studio Community Installer, selezionando nei

workloads tutto ciò relativo alla programmazione in C++. Soltanto ora creeremo un nuovo ambiente virtuale (lo chiameremo *magenta_env* ad esempio). Servirà aprire il terminale e impartire questi comandi:

```
conda create -n magenta_env python=3.7.9 -y
conda activate magenta_env
pip install tensorflow-gpu==2.0.0
```

Da questo momento l'ambiente virtuale Conda è stato creato ed è pronto per l'installazione di Magenta. *Senza chiudere il terminale*, ancora attivo nell'ambiente (*magenta_env*), impartiamo il comando:

```
pip install magenta
```

Al termine del comando Magenta sarà correttamente installato e ne potremo usare gli strumenti che mette a disposizione. La Magenta MIDI Interface può essere invocata con il comando "**magenta_midi**". Per avere una lista delle modalità di utilizzo disponibili per l'interfaccia impartiamo il comando:

```
magenta_midi --helpfull
```

Le opzioni disponibili permettono di impostare, oltre alle porte di input/output e la rete neurale di riferimento, parametri come *il segnale MIDI per cambiare la velocità del metronomo*, *il segnale MIDI di Panic Button* (interrompe tutte le note correntemente attive), *la velocità fissa del metronomo*, *abilitare o meno il metronomo*, *inviare in loop la risposta generata* e tanto altro ancora. Nelle nostre interazioni decideremo di usare le impostazioni di default, dove il metronomo è abilitato ed impostato a *120 BPM* (Beats per Minute), non ci sono Panic Buttons né altri Control Buttons (in altri termini, *invieremo note e basta*)

```
it off.
(an integer)
--max_listen_ticks_control_number: The control change number to use for
controlling maximum listen duration. The value for the control number will
be used in clock ticks. After this number of ticks, a response will
automatically be generated. A 0 value signifies infinite duration.
(an integer)
--metronome_channel: The 0-based MIDI channel to output the metronome on.
Ignored if 'enable_metronome' is False or 'clock_control_number' is
provided.
(default: '1')
(an integer)
--min_listen_ticks_control_number: The control change number to use for
controlling minimum listen duration. The value for the control number will
be used in clock ticks. Inputs less than this length will be ignored.
(an integer)
--mutate_control_number: The control change number to use with value 127 as a
mutate signal to generate a new response using the current response sequence
as a seed.
(an integer)
--output_ports: Comma-separated list of names of the output MIDI ports.
(default: 'magenta_out')
--panic_control_number: The control change number to use with value 127 as a
panic signal to close open notes and clear playback sequence.
(an integer)
```

Figura 5.7: Lista parziale delle opzioni della Magenta MIDI Interface

5.4 Interazione con Magenta

Da questo momento si suppone che Magenta sia correttamente installato nell'ambiente virtuale creato con Conda. È bene ricordare che da questo momento in poi ogni interazione dovrà essere preceduta dall'attivazione dell'ambiente virtuale appena creato con il comando:

```
conda activate magenta_env
```

Ricordiamo che per avviare l'interazione con la Magenta MIDI Interface è necessario che siano pronti all'uso almeno un dispositivo di input MIDI, un dispositivo di output MIDI ed un Tensorflow model. Essendo presenti sul GitHub ufficiale di Magenta una *vasta gamma di modelli già preaddestrati* non è stato difficile avviare una prima interazione potendone semplicemente sceglierne uno. Per tutte le interazioni successivamente descritte è stato usato il modello **Attention RNN**, appartenente alla categoria Melody RNN. Tutte i modelli appartenenti alla categoria Melody RNN sono progettati come reti neurali *LSTM (Long Short Term Memory)* a due livelli. Le altre reti neurali alla categoria sono *Mono RNN*, *Basic RNN* e *Lookback RNN*. Per le interazioni è stato scelto Attention RNN in quanto introduce il concetto di "attenzione" che permette al modello di accedere più facilmente alle informazioni passate senza dover salvare tali informazioni nelle celle di stato della rete neurale [31]. Dal punto di vista della performance Attention RNN è il modello che meglio simula il concetto di **botta e**

risposta; grazie all'attenzione *ragiona meglio* sulla struttura della chiamata musicale per generare una risposta dal piacevole ascolto.

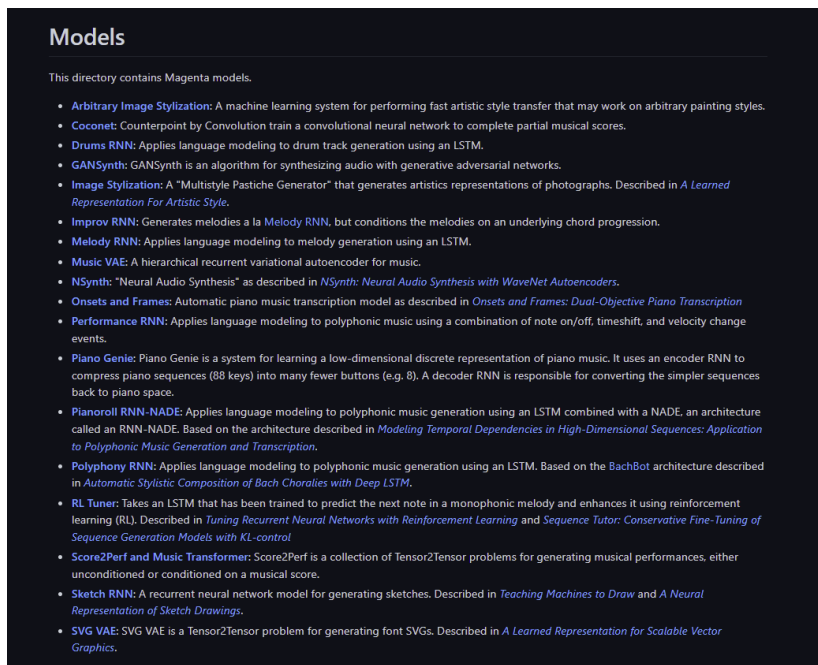


Figura 5.8: Lista dei modelli Tensorflow preaddestrati

5.4.1 UMX-25 & Magenta

La prima interazione testata ha cercato di far interagire Magenta con il controller **UMX-25** [32] già disponibile nel laboratorio di Computer Music.



Figura 5.9: UMX-25 MIDI Controller

Come dispositivo di uscita è stata scelta una porta virtuale ad hoc per il testing (che chiameremo *TestingPort*). Per assicurarci che le porte siano disponibili e pronte a farle interagire con la Magenta MIDI Interface è consigliato eseguire il comando:

```
magenta_midi --list_ports
```

Da questo comando sarà possibile leggere le porte di input ed output pronte ad interagire con Magenta. Ci sono degli aspetti importanti da notare:

- Le porte sia di input che di output sono automaticamente enumerate dall'interfaccia, il loro nuovo nome sarà costituito dal loro nome precedente seguito dall'indice generato dall'interfaccia, per poter far riferimento ad una porta è necessario specificare il nome della porta per intero (compreso di indice).
- Ogni porta virtuale avrà una corrisponde porta di input ed output. Ad esempio è possibile osservare come la porta "MarcoSmiles" sia presente sia tra le porte di input che quelle di output.

```
1 (magenta_env) C:\Users\puddi>magenta_midi --list_ports
2 Instructions for updating:
3 non-resource variables are not supported in the long term
4 Input ports: 'MarcoJams 1', 'MarcoSmiles 0'
5 Output ports: 'MarcoJams 2', 'MarcoSmiles 1', 'Microsoft GS Wavetable Synth 0'
6
```

È bene non confondersi tra le porte perchè l'avvio di un'interazione sbagliata è *fatale* per il terminale, servirà riavviarlo per poter riutilizzare la Magenta MIDI Interface.

- *Il ciclo di vita delle porte virtuali* corrisponde con *il ciclo di vita del software che le gestisce* (come ad esempio *loopMIDI*). Ciò significa che per interagire con le porte virtuali bisogna **assicurarsi** che *loopMIDI* sia avviato e gestisca tali porte.

Per avviare l'interazione è stato usato il comando

```
magenta_midi \  
--input_ports="INPUT_PORTS" \  
--output_ports="OUTPUT_PORTS" \  
--bundle_files="BUNDLE_PATHS"
```

dove nelle porte di input è stato specificato "UMX-25", nelle porte di output "TestingPort" e nei bundle files è stato specificato il file "attention_rnn.mag", che corrisponde al file del modello Tensorflow prelevato direttamente dalla lista dei modelli preaddestrati disponibili (i modelli si presenteranno sempre con l'estensione **.mag**). Il comando per intero è quindi:

```
magenta_midi \  
--input_ports="UMX-25 0" \  
--output_ports="TestingPort 2" \  
--bundle_files="attention_rnn.mag"
```

Dopo pochi secondi dall'avvio del comando sul terminale comparirà:

```
1 (magenta_env) C:\Users\puddi\Desktop\fluid>magenta_midi \  
2 --input_ports="UMX-25 0" \  
3 --output_ports="TestingPort 2" \  
4 --bundle_files="attention_rnn.mag"  
5  
6 [...]
7
8 Instructions:
9 Start playing when you want to begin the call phrase.
10 When you want to end the call phrase, stop playing and wait one clock tick.
11 Once the response completes, the interface will wait for you to begin playing again to
12 start a new call phrase.
13
14 To end the interaction, press CTRL-C.
15
```

In questa interazione non stati riscontrati grossi problemi ed in breve è stato possibile osservare per la prima volta la Magenta MIDI Interface all'opera. Per osservare i messaggi generati da Magenta apriamo MidiView e selezioniamo la porta di output dalla tendina in alto al centro. Non avendo specificato opzioni in particolare, il metronomo nell'interazione è abilitato ed è riprodotto con la sequenza di note **B0 - B0 - B0 - G#1, Velocity 64, Canale 2**.

Premendo i tasti di UMX-25 e attendendo poche battute, è possibile leggere sempre da MidiView la response generata. Per poter invece *ascoltare* la response basterà aprire un DAW, come *FLStudio*, creare un synth qualsiasi e collegarlo alla porta virtuale scelta come output di Magenta.

Time	Dir	Message	Channel	Value	Hex
15:07:56.320	out	Note on B0 Velocity 64	2	64	91 23 40
15:07:56.370	out	Note off B0 Velocity 64	2	64	81 23 40
15:07:56.820	out	Program change 117	2		C1 75
15:07:56.820	out	Note on G#1 Velocity 64	2	64	91 2C 40
15:07:56.871	out	Note off G#1 Velocity 64	2	64	81 2C 40
15:07:57.320	out	Note on B0 Velocity 64	2	64	91 23 40
15:07:57.370	out	Note off B0 Velocity 64	2	64	81 23 40
15:07:57.820	out	Note on B0 Velocity 64	2	64	91 23 40
15:07:57.870	out	Note off B0 Velocity 64	2	64	81 23 40
15:07:58.320	out	Note on B0 Velocity 64	2	64	91 23 40
15:07:58.370	out	Note off B0 Velocity 64	2	64	81 23 40
15:07:58.820	out	Note on G#1 Velocity 64	2	64	91 2C 40
15:07:58.820	out	Program change 117	2		C1 75
15:07:58.870	out	Note off G#1 Velocity 64	2	64	81 2C 40
15:07:59.320	out	Note on B0 Velocity 64	2	64	91 23 40
15:07:59.370	out	Note off B0 Velocity 64	2	64	81 23 40

Figura 5.10: Metronomo di Magenta letto da MidiView

5.4.2 Errori riscontrati con l'uso delle porte virtuali

Il passo successivo, ora che la Magenta MIDI Interface è stata testata con un controller fisico in input, è quello di **usare MarcoSmiles come porta di input**. Come è già stato specificato precedentemente le porte virtuali sono individuate da Magenta come porte disponibili *sia per l'input che per l'output*. Una prima serie di tentativi errati di interazione tra MarcoSmiles e Magenta hanno portato alla miglior comprensione della MIT (Magenta MIDI Interface). In particolare l'interazione è stata avviata usando come porta di input e di output *la stessa porta virtuale* (MarcoSmiles 0 per l'input, MarcoSmiles 1 per l'output) sotto l'ipotesi che le porte virtuali corrispondessero ad un interfaccia MIDI virtualizzata dove l'output è *ben distinto* dall'input (esattamente come un dispositivo fisico). I primi tentativi sono stati quindi avviati con il comando:

```
magenta_midi \
--input_ports="MarcoSmiles 0" \
--output_ports="MarcoSmiles 1" \
--bundle_files="attention_rnn.mag"
```

Dopo pochi secondi dal comando l'interazione verrà avviata con successo dal terminale ma dopo appena pochi istanti loopMIDI *notificherà un errore di throughput*.

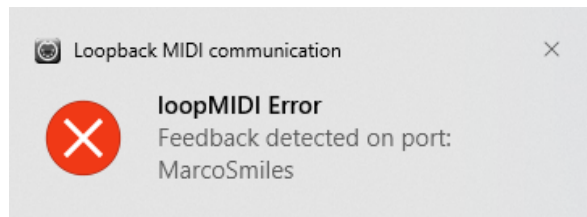


Figura 5.11: Errore riscontrato da loopMIDI

Osservando meglio sia il *throughput su loopMIDI* che il *flusso dei messaggi via Mi-diView* è stato possibile osservare come in realtà, avviando l'interazione specificando su questo schema, il risultato ottenuto è ben diverso da quello atteso (cioè quello dell'interazione dove la call e la response sono ben distinte su versioni distinte della stessa porta).

Avviando l'interazione, la prima nota del metronomo (cioè **B0**) inviata dalla MIT sulla porta di output MarcoSmiles viene in realtà interpretata *anche* come nota in input all'interfaccia, quindi MIT non distingue le note da esse generate da quelle fornite in input (come invece ipotizzato in precedenza). Pertanto la nota generata da MIT stessa, interpretata anche come input, viene rifornita in output creando un ciclo infinito che aumenta *vertiginosamente* il throughput sulla porta virtuale selezionata nell'interazione;

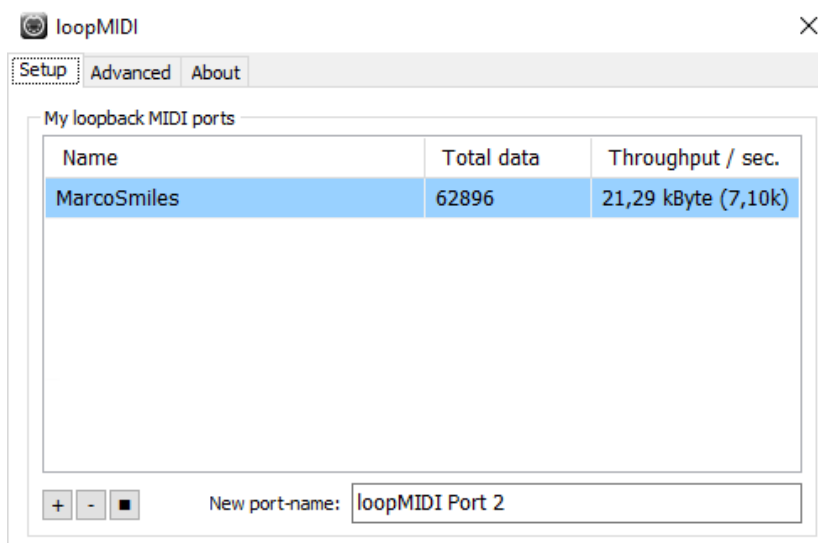


Figura 5.12: Throughput anomalo sulla porta MarcoSmiles

loopMIDI, osservando un **throughput anomalo** (è impossibile raggiungere un throughput tale da una performance umana) muta la porta virtuale. L'interazione da terminale non è interrotta né individua errori ma è *idle* poichè essendo mutata la porta, nessuna nota (neanche il metronomo) attraversa l'interfaccia.

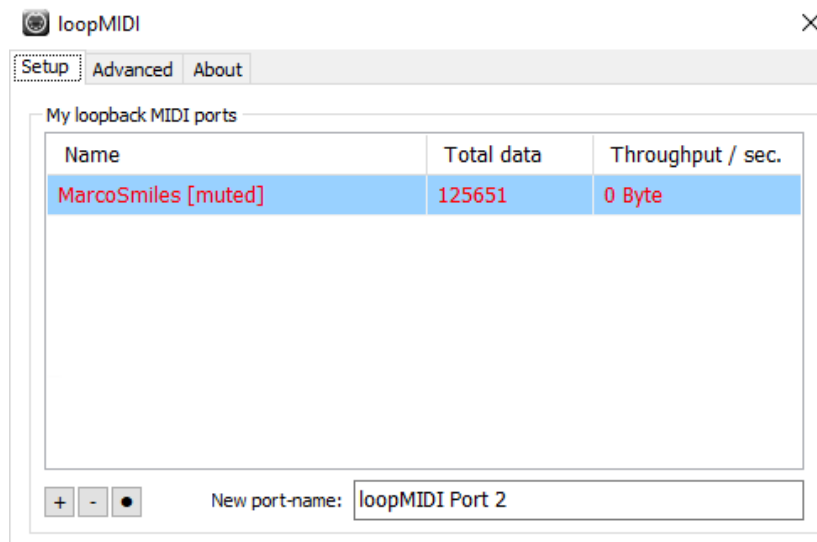


Figura 5.13: MarcoSmiles mutata da loopMIDI

Con questi tentativi di interazione e il problema individuato è stato possibile *rompere il modello mentale* del funzionamento della MIT costruito fino ad allora e *comprenderne il vero funzionamento*. La Magenta MIDI Interface invia sulle porte di output specificate, per tutta la durata dell'interazione:

- **Le note ricevute** dalle porte di input selezionate nell'interazione
- **Il metronomo**, se abilitato
- **Le note generate** dalla rete neurale sulla base del modello selezionato

Essendo quindi *questo* il modello di funzionamento della MIT è strettamente necessario distinguere la risposta di Magenta (composta dai tre elementi sovraelencati) dalle note inviate dai dispositivi di input. La figura 5.14 mostra come, sulla stessa porta monitorata su MidiView, la MIT invii sia le note inviate in input (in **rosso**), che il metronomo (in **blu**) che le note generate (in **verde**)

Time	Dir	Message	Channel	Value	Hex
17:08:57.381	out	Note off C2 Velocity 0	1	0	90 30 00
17:08:57.567	out	Note on B0 Velocity 64	2	64	91 23 40
17:08:57.605	out	Note on C2 Velocity 104	1	104	90 30 68
17:08:57.618	out	Note off B0 Velocity 64	2	64	81 23 40
17:08:57.684	out	Note off C2 Velocity 0	1	0	90 30 00
17:08:57.820	out	Note on C2 Velocity 104	1	104	90 30 68
17:08:57.924	out	Note off C2 Velocity 0	1	0	90 30 00
17:08:58.067	out	Note on B0 Velocity 64	2	64	91 23 40
17:08:58.100	out	Note on C2 Velocity 104	1	104	90 30 68
17:08:58.118	out	Note off B0 Velocity 64	2	64	81 23 40
17:08:58.172	out	Note off C2 Velocity 0	1	0	90 30 00
17:08:58.356	out	Note on C2 Velocity 104	1	104	90 30 68
17:08:58.420	out	Note off C2 Velocity 0	1	0	90 30 00
17:08:58.567	out	Note on G#1 Velocity 64	2	64	91 2C 40
17:08:58.567	out	Program change 117	2		C1 75
17:08:58.618	out	Note off G#1 Velocity 64	2	64	81 2C 40

Time	Dir	Message	Channel	Value	Hex
17:09:03.117	out	Note off B0 Velocity 64	2	64	81 23 40
17:09:03.567	out	Note on B0 Velocity 64	2	64	91 23 40
17:09:03.618	out	Note off B0 Velocity 64	2	64	81 23 40
17:09:04.068	out	Note on B0 Velocity 64	2	64	91 23 40
17:09:04.118	out	Note off B0 Velocity 64	2	64	81 23 40
17:09:04.389	out	Program change 117	2		C1 75
17:09:04.568	out	Note on G#1 Velocity 64	2	64	91 2C 40
17:09:04.568	out	Program change 117	2		C1 75
17:09:04.618	out	Note off G#1 Velocity 64	2	64	81 2C 40
17:09:04.693	out	Note on C2 Velocity 100	1	100	90 30 64
17:09:04.833	out	Note off C2 Velocity 64	1	64	80 30 40
17:09:04.943	out	Note on A1 Velocity 100	1	100	90 2D 64
17:09:05.067	out	Note on B0 Velocity 64	2	64	91 23 40
17:09:05.068	out	Note off A1 Velocity 64	1	64	80 2D 40
17:09:05.118	out	Note off B0 Velocity 64	2	64	81 23 40
17:09:05.209	out	Note on G1 Velocity 100	1	100	90 2B 64

Figura 5.14: Call e Response viste da MidiView

Un modo immediato per ottenere questa separazione è specificare una *nuova porta virtuale* designata solo ed esclusivamente ad ospitare la risposta della Magenta MIDI Interface. Da qui nasce il termine "MarcoJams", nome della nuova porta virtuale.

5.4.3 MarcoSmiles & Magenta

Dimostrato il motivo per cui è strettamente necessario distinguere la risposta dalle note inviate alla MIT, creiamo la *nuova* porta virtuale "MarcoJams" su loopMIDI.

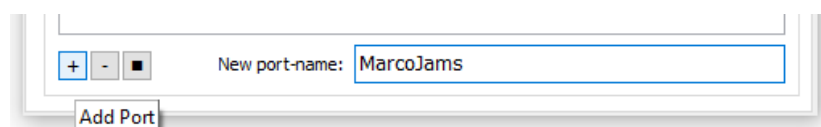


Figura 5.15: Creazione della nuova porta virtuale

Ora che sono pronte all'uso sia la porta MarcoSmiles che la porta MarcoJams avviamo l'interazione usandole rispettivamente come porte di input ed output. Dopo aver eseguito controllato la loro enumerazione stampando la lista delle porte disponibili per la MIT, impartiamo il comando:

```
magenta_midi \  
--input_ports="MarcoSmiles 0" \  
--output_ports="MarcoJams 2" \  
--bundle_files="attention_rnn.mag"
```

Dopo qualche secondo MarcoSmiles sarà attraversata *solo dalle note intenzionalmente inviate in input* all'interfaccia mentre MarcoJams sarà attraversata dalle *note in input, il metronomo e la risposta dell'interfaccia*. Aprendo il software MarcoSmiles, collegando il Leap Motion Controller e assicurandosi che il tasto MIDI ON sia attivo si otterrà la desiderata interazione tra MarcoSmiles e Magenta! Per ascoltare i risultati dell'interazione basterà seguire gli stessi passaggi descritti nell'interazione con UMX-25, collegando questa volta al sintetizzatore virtuale la porta "MarcoJams".

5.5 Modifiche all'interfaccia grafica di MarcoSmiles

Ad ora è possibile avviare il duetto tra MarcoSmiles e Magenta soltanto in *maniera manuale*; l'utente deve, in due momenti separati, avviare MarcoSmiles, abilitare il MIDI e soltanto dopo aprire manualmente il prompt dei comandi ed avviare l'interazione seguendo le istruzioni descritte precedentemente.

L'obiettivo successivo è quindi fornire all'interfaccia grafica di MarcoSmiles un **modulo**, apprezzabile in termini grafici e di usabilità, che possa permettere all'utente di automatizzare questo compito, cioè avviare e terminare l'interazione *senza doversi sporcare le mani ad impartire comandi al terminale*. L'idea è quella di **arricchire la Play Scene** aggiungendo una nuova area di bottoni esclusivamente dedicata all'interazione con Magenta, fornire a questi bottoni il comportamento tramite script e assicurarsi del corretto funzionamento.

5.5.1 GameObjects

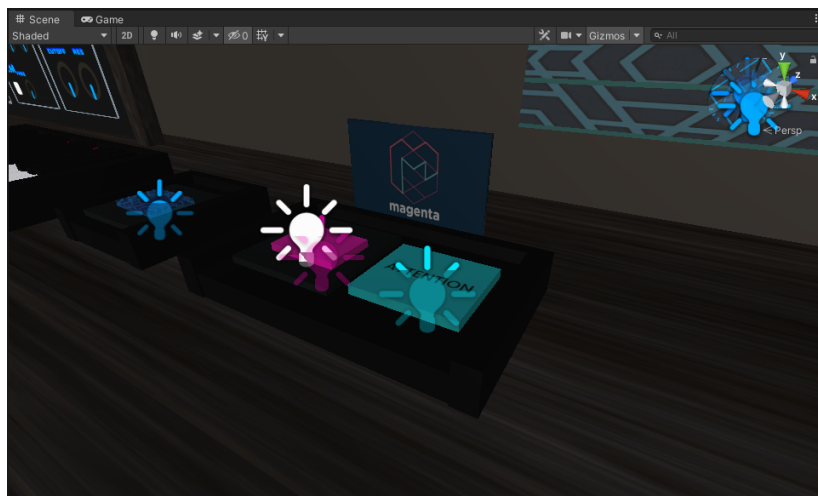


Figura 5.16: Nuovi bottoni nella Play Scene

È stato aggiunto un nuovo **modulo di bottoni**, contraddistinti dal logo di Magenta, non invasivo sull'interfaccia e facilmente individuabile. Il bottone ciano posto sulla destra nel modulo è un *toggle-button*, permette di cambiare (non a run-time bensì prima dell'avvio dell'interazione) il modello Tensorflow da utilizzare nell'interazione;



Figura 5.17: Nuova interfaccia dalla vista dell'utente

di default è selezionato il modello **Attention RNN**, pigiandolo verrà selezionato il modello **Lookback RNN**. È stata data la possibilità di cambiare tra questi modelli ma è bene ricordarsi che non sono gli unici disponibili, la scelta di questi due in

particolare è dettata dal fatto che sono i due modelli più *"orecchiabili"* appartenenti alla stessa famiglia delle *Melody RNN*. Il pulsante color magenta avvia l'interazione tra MarcoSmiles e Magenta, fornendo l'output sulla porta MarcoJams e usando il modello selezionato sul bottone color ciano. Automatizza l'avvio dell'interazione controllando che le porte siano attive e pronte all'uso. Sia in esito positivo che in esito negativo il bottone fornisce **feedback grafico**, in modo che l'utente possa riconoscere (senza essere necessariamente esperto) quando l'interazione è *attiva*, *spenta*, *disponibile* o *meno*.

5.5.2 Scripts

Il click del bottone magenta sollecita l'avvio delle procedure utili a determinare la *possibilità di avvio dell'interazione*; se le condizioni per avviare l'interazione dovessero essere valide al momento del click del bottone, lo script *apre il terminale* ed *avvia l'interazione con la Magenta MIDI Interface*.

```
1  if (toggle == 0)
2      {
3          if (canStartMagenta())
4              {
5                  StartCoroutine("buttonBlink");
6                  Debug.Log("AVVIANDO MAGENTA! "+RNNToggle);
7                  toggle = 1;
8                  interaction();
9              }
10         else
11             {
12                 StartCoroutine("errorBlink");
13             }
14     }
15     else if (toggle == 1)
16     {
17         if (buttonBlinking)
18             {
19                 StopCoroutine("buttonBlink");
20                 buttonBlinking = false;
21             }
22         if (!interactionProcess.HasExited)
23             {
24                 interactionProcess.Kill();
25                 interactionProcess.WaitForExit();
26                 interactionProcess.Dispose();
```

```

27     }
28     resetButton();
29     toggle = 0;
30 }
31
32 }
33

```

Listing 5.1: MagentaButtonScript.buttonClicked

Concorrentemente all'avvio dell'interazione il bottone **lampeggerà di magenta** per tutta la durata della performance; in questo modo l'utente sa che l'interazione è in *fase di avvio*:

```

1  IEnumerator buttonBlink() {
2      int blink = 0;
3      float blinkInterval = 0.5f;
4      buttonBlinking = true;
5      while (true)
6      {
7          if (blink == 0)
8          {
9              Material buttonMaterial = GetComponent<Renderer>().material;
10             buttonMaterial.SetColor("_EmissionColor", Color.magenta);
11             buttonLight.intensity = 2;
12             blink = 1;
13             yield return new WaitForSeconds(blinkInterval);
14         }
15         else if (blink == 1)
16         {
17             Material buttonMaterial = GetComponent<Renderer>().material;
18             buttonMaterial.SetColor("_EmissionColor", Color.black);
19             buttonLight.intensity = 0;
20             blink = 0;
21         }
22     }
23 }
24

```

Listing 5.2: MagentaButtonScript.buttonBlink

Se l'avvio dell'interazione non dovesse essere possibile (perché le porte non sono disponibili o perché l'utente ha interrotto manualmente la fase di controllo), il bottone **lampeggerà di rosso** per pochi istanti, tempo sufficiente per richiamare l'attenzione

```

1  IEnumerator errorBlink ()
2  {
3      Material originalMaterial = GetComponent<Renderer>().material;
4      Color originalButtonLightColor = buttonLight.color;
5      buttonLight.color = Color.red;
6      GetComponent<Renderer>().material.color = Color.red;
7      for (int i=0;i<3;++i)
8      {
9          GetComponent<Renderer>().material.SetColor("_EmissionColor", Color.red);
10         buttonLight.intensity = 2;
11         yield return new WaitForSeconds(0.2f);
12         GetComponent<Renderer>().material.SetColor("_EmissionColor", Color.black);
13         buttonLight.intensity = 0;
14         yield return new WaitForSeconds(0.2f);
15     }
16     GetComponent<Renderer>().material.color = Color.magenta;
17     buttonLight.color = originalButtonLightColor;
18 }
19

```

Listing 5.3: MagentaButtonScript.canStartMagenta

Il controllo delle condizioni per l'avvio dell'interazione è fornito dalla funzione *canStartMagenta()*, che avvia un prompt dei comandi il cui input e output è *ridirezionato* in modo da poter essere analizzato.

```

1  private bool canStartMagenta ()
2  {
3
4      bool input_port_available = false;
5      bool output_port_available = false;
6
7      //Sostituire questo path con il PROPRIO path ad Anaconda
8      string pathToConda = "C:\\Users\\puddi\\anaconda3\\Scripts\\activate.bat";
9
10     string input_port_name;
11     string output_port_name;
12
13     string path = FileUtils.GeneratePath("..");
14     Debug.Log("Path generated: " + path);
15     seekProcess = new Process
16     {
17         StartInfo = new ProcessStartInfo
18         {
19             FileName = "cmd.exe",
20             RedirectStandardInput = true,

```

```

21         UseShellExecute = false,
22         RedirectStandardOutput = true,
23         WorkingDirectory = path
24     }
25 };
26 seekProcess.Start();
27 using (var sw = seekProcess.StandardInput)
28 {
29     if (sw.BaseStream.CanWrite)
30     {
31         sw.WriteLine(pathToConda);
32         // Attiva l'environment
33         sw.WriteLine("activate gputest");
34         // Stampando la lista degli ambienti Conda attivi e pronti all'uso
35         sw.WriteLine("conda env list");
36         //Stampando la lista delle porte MIDI disponibili per Magenta
37         sw.WriteLine("magenta_midi --list_ports");
38     }
39 }
40
41 //L'output ridirezionato viene analizzato per determinare se le porte MIDI sono pronte
42 while (!seekProcess.StandardOutput.EndOfStream)
43 {
44     var line = seekProcess.StandardOutput.ReadLine();
45     Debug.Log("### - MAGENTA BUTTON - ### | " + line);
46     if (line.Contains("Input ports"))
47     {
48         if (line.Contains("MarcoSmiles"))
49         {
50             int index = line.IndexOf("MarcoSmiles");
51             input_port_name = line.Substring(index, 13);
52             Debug.Log("PORTA DI INPUT PRONTA: " + input_port_name);
53             input_port_available = true;
54         }
55         else
56         {
57             Debug.Log("PORTA DI INPUT NON PRONTA");
58         }
59     }
60     if (line.Contains("Ouput ports"))
61     {
62         if (line.Contains("MarcoJams"))
63         {
64             int index = line.IndexOf("MarcoJams");
65             output_port_name = line.Substring(index, 11);
66             Debug.Log("PORTA DI OUTPUT PRONTA: " + output_port_name);
67             output_port_available = true;

```

```

68         }
69         else
70         {
71             Debug.Log("PORTA DI OUTPUT NON PRONTA");
72         }
73     }
74 }
75
76 if (input_port_available && output_port_available)
77 {
78     Debug.Log("Tutto pronto!");
79     return true;
80 }
81 return false;
82 }
83

```

Listing 5.4: MagentaButtonScript.buttonClicked

L'interazione vera e propria è ottenuta eseguendo un file **.bat** contenente le istruzioni in sequenza per avviare l'interazione. Il click del bottone ciano sollecita la funzione *giantClicked()*; per dare un miglior feedback grafico il bottone varia la propria altezza a seconda del proprio stato:

```

1  public void giantClicked ()
2  {
3      if (!RNNToggle)
4      {
5          RNNToggle = true;
6          Debug.Log("RNN is now " + RNNToggle);
7          transform.Translate(new Vector3(0, -0.05f, 0));
8          GetComponent<Renderer>().material = lookbackMaterial;
9      } else {
10         RNNToggle = false;
11         Debug.Log("RNN is now " + RNNToggle);
12         transform.Translate(new Vector3(0, 0.05f, 0));
13         GetComponent<Renderer>().material = attentionMaterial;
14     }
15 }
16

```

Listing 5.5: MagentaButtonScript.giantClicked

5.6 Esempio di interazione completa

Ora che è tutto pronto, può essere mostrata la sequenza di passaggi per raggiungere un'interazione completa.

1. Avviare **loopMIDI**, assicurarsi che esistono le porte virtuali *MarcoSmiles* e *MarcoJams*.
2. Avviare MarcoSmiles, scegliere il dataset per la performance, collegare il Leap Motion Controller, entrare nella Play Scene.
3. Premere il bottone **MIDI ON**, scegliere tra i due modelli disponibili (Attention o Lookback) tramite il *pulsante ciano*, premere il *pulsante magenta*.
4. *Attendere l'avvio dell'interazione*. Il tempo di attesa dipende strettamente dalle performance della macchina.



Figura 5.18: Interazione in corso

5. Avviare un **DAW** (e.g. *FLStudio* [33]), selezionare un synth, collegare al synth la porta MIDI MarcoJams. Se non si ha a disposizione un DAW è possibile utilizzare anche un semplice sintetizzatore, come *FluidSynth* [34].
6. **(Importante)** *Divertirsi*.

Capitolo 6

Conclusioni

6.1 Considerazioni

L'obiettivo comune degli sviluppatori di MarcoSmiles, coloro che hanno già apportato modifiche al software o che ne apporteranno in futuro, è in generale quello di arricchire l'esperienza di utilizzo. L'obiettivo preposto è stato decisamente raggiunto. L'arricchimento dell'esperienza è stata raggiunto raggiungendo, in successione, i due seguenti obiettivi:

- **Ottenimento dell'interazione** tra MarcoSmiles e la Magenta MIDI Interface, strumento offerto da Magenta by Google Brain Team. Sono stati effettuati diversi test i quali hanno avuto sempre esito positivo; l'avvio dell'interazione non genera errori e i messaggi inviati sulla porta MarcoSmiles vengono sempre ricevuti dall'interfaccia, ai quali risponde dopo circa 2 battute.
- **Integrazione di un nuovo modulo grafico** nell'interfaccia di MarcoSmiles per permettere l'automatizzazione dell'interazione con Magenta MIDI Interface. Il modulo è correttamente integrato nell'interfaccia, il comportamento osservato utilizzando il modulo corrisponde al comportamento atteso e ideato in fase di design; grazie al nuovo modulo integrato anche l'utente meno esperto può divertirsi a duettare con la Magenta MIDI Interface.

6.2 Sviluppi futuri

Come già rimarcato, l'obiettivo degli sviluppatori di MarcoSmiles è arricchire il software di nuove idee, moduli, apportare miglioramenti. In questa sezione verranno analizzate alcune possibilità, spunti, idee per nuove funzionalità da implementare all'interno del sistema:

- **Porting di MarcoSmiles su microcontrollore:** attualmente MarcoSmiles è un software generalmente installato su PC o Desktops, genera eventi MIDI su una porta virtuale. L'unico modo attualmente disponibile per ascoltare gli eventi MIDI generati da MarcoSmiles è avviare una Digital Audio Workstation (come FL Studio) e collegare ad un sintetizzatore o ad un qualsiasi altro sound-generator la porta virtuale MarcoSmiles. Si potrebbe pensare di traslare l'intero funzionamento di MarcoSmiles su un microcontrollore come *Arduino* [35] o *Raspberry PI* [36] in modo da ottenere un dispositivo MIDI indipendente, come se fosse un vero e proprio strumento musicale, direttamente integrabile in una MIDI network tramite USB (o MIDI Legacy, se necessario).
- **Suonare più note contemporaneamente:** attualmente il sistema non prevede la polifonia, permette di associare ad ogni posizione delle mani una sola nota. Per ottenere la polifonia si potrebbe pensare ad un nuovo tipo di associazione, dove ad ogni posizione delle mani è associato un accordo piuttosto che una singola nota.
- **MarcoSmiles VR:** Unity3D, motore grafico utilizzato per lo sviluppo di MarcoSmiles, è frequentemente utilizzato anche in ambito di Realtà Virtuale. Per ottenere un'esperienza ancora più coinvolgente si potrebbe pensare di creare una versione di MarcoSmiles VR, dove l'esperienza di utilizzo richiede l'uso di un visore (come un *Oculus Rift* [37]), più sensori e il Leap Motion Controller.
- **Training via Cloud Computing:** gli studi e gli esperimenti effettuati da Daniele Salerno mostrano come le workstation comunemente disponibili agli utenti non dispongano di potenza di calcolo sufficiente ad ultimare la fase di training in tempi inferiori all'ordine delle decine di minuti (fino ad più di un'ora per datasets

complessi). Potrebbe essere sfruttata la potenza di calcolo offerta da servizi di cloud computing per creare nuovi datasets in tempi decisamente inferiori.

- **Leap Motion come unico controller:** attualmente MarcoSmiles richiede l'utilizzo di un mouse per la selezione degli elementi sull'interfaccia; il Leap Motion è richiesto esclusivamente durante le fasi di Training e Performance. Ai fini di rendere il software sia più semplice da utilizzare che più accessibile, si potrebbe pensare di permettere all'utente di usare il Leap Motion anche per selezionare elementi sull'interfaccia e navigare nel software, così da usare il Leap Motion Controller come unico dispositivo.
- **Switchare tra datasets durante la performance:** tra musicisti è abbastanza comune saper suonare più strumenti e divertirsi, durante la stessa performance, ad utilizzare strumenti diversi. Attualmente MarcoSmiles è in grado di avviare la fase di performance seguendo le regole di un solo dataset selezionato; per cambiare dataset è strettamente necessario interrompere la performance, scegliere il nuovo dataset ed ripartire da capo. Sarebbe divertente permettere, quando si hanno a disposizione più datasets (*un dataset può essere visto come uno strumento perché detta le regole di associazione tra posizione delle mani e nota generata*) di switchare tra i datasets disponibili senza interrompere la performance.

Bibliografia

- [1] *Bell Laboratories*. <https://www.bell-labs.com>.
- [2] *Leap Motion Controller*. <https://www.ultraleap.com/product/leap-motion-controller/>.
- [3] *Magenta by Tensorflow*. <https://magenta.tensorflow.org>.
- [4] *A.I. DUET by Google Creative Lab*. <https://experiments.withgoogle.com/ai/ai-duet/view/>.
- [5] *Botta e Risposta*. [https://it.wikipedia.org/wiki/Botta_e_risposta_\(musica\)](https://it.wikipedia.org/wiki/Botta_e_risposta_(musica)).
- [6] *Theremin*. <https://it.wikipedia.org/wiki/Theremin>.
- [7] *Moog Music - Theremini*. <https://www.moogmusic.com/products/etherwave-theremins/theremini>.
- [8] *Neural Arpeggiator*. <https://codepen.io/teropa/pen/ddqEwj>.
- [9] *#MadeWithMagenta*. <https://magenta.tensorflow.org/demos/community/>.
- [10] *KAiKU Music Gloves*. <https://www.kaikumusicgloves.com/>.
- [11] *Magenta Studio*. <https://magenta.tensorflow.org/studio/>.
- [12] *Ableton Live*. <https://www.ableton.com/en/live/>.
- [13] *Virtual Piano*. <https://www.gallery.leapmotion.com/virtual-piano-for-beginners>.
- [14] *GECO MIDI*. <https://gallery.leapmotion.com/geco-midi/>.
- [15] *MIDI*. https://it.wikipedia.org/wiki/Musical_Instrument_Digital_Interface.
- [16] *Sequential*. <https://www.sequential.com>.
- [17] *loopMIDI*. <https://www.tobias-erichsen.de/software/loopmidi.html>.
- [18] *Magenta on GitHub*. <https://github.com/magenta/magenta>.
- [19] *Tensorflow*. <https://www.tensorflow.org>.
- [20] *'Magenta' Is Google's New Project To Make Art With Artificial Intelligence*. <https://www.popsci.com/magenta-is-googles-project-to-make-art-with-artificial-intelligence/>.
- [21] *Magenta Pre-trained Models and API*. <https://github.com/magenta/magenta/tree/main/magenta/models>.
- [22] *Project Magenta: Music and Art with Machine Learning (Google I/O '17)*. https://www.youtube.com/watch?v=2FAjQ6R_bf0.

- [23] *Unity 3D*. <https://unity.com>.
- [24] *Midi View*. <https://hautetechnique.com/midi/midiview/>.
- [25] *MIDI Tools*. <https://mountainutilities.eu/miditools>.
- [26] *Conda*. <https://docs.conda.io/en/latest/>.
- [27] *Conda Installation*. <https://docs.conda.io/projects/conda/en/latest/user-guide/install/>.
- [28] *NVIDIA cuDNN*. <https://developer.nvidia.com/cudnn>.
- [29] *NVIDIA CUDA*. <https://developer.nvidia.com/cuda-toolkit>.
- [30] *NVIDIA CUDA Toolkit*. <https://developer.nvidia.com/cuda-10.0-download-archive>.
- [31] *Music Generation Using Neural Networks*. <http://cs229.stanford.edu/proj2016/report/Lou-MusicGenerationUsingNeuralNetworks-report.pdf>.
- [32] *Behringer UMX 25*. <https://www.behringer.com/product.html?modelCode=P0447>.
- [33] *FLStudio*. <https://www.image-line.com>.
- [34] *FluidSynth*. <https://www.fluidsynth.org>.
- [35] *Sito ufficiale di Arduino*. <https://www.arduino.cc>.
- [36] *Sito ufficiale di Raspberry PI*. <https://www.raspberrypi.com>.
- [37] *Oculus Rift S*. <https://www.oculus.com/rift-s>.

Un ringraziamento sentito va al Prof. *Rocco Zaccagnino*, mio relatore nonché grande entusiasta dell'*informatica*, della *musica* e della *matematica*; un'anima affine che ha saputo sempre incoraggiarmi durante questi difficili mesi. Gli sono davvero grato dell'amicizia che ha mostrato e della fiducia che ha sempre trasmesso.

Un grazie di cuore va poi a loro quattro, in nessun ordine particolare: *A. Gravino*, *C. Fabbri*, *R. Zheng* e *C. Napolitano*. Non oserò chiedervi la parola che squadri da ogni lato il vostro animo informe. Noi cinque come quelle di una stessa mano.

Vorrei inoltre trovare l'occasione di ringraziare *G. Di Mauro* per esser sempre stato disponibile, pronto ad aiutarmi quando in difficoltà, di buon cuore, mai agitato, *l'emblema della calma*. Con te ho imparato prendere le cose *un po' più alla leggera*.

Infine vorrei ringraziare *Sara*, la mia *mosca*, colei che ha sempre sostenuto la mia ricerca, la mia curiosità. Tante altre *scale* con te, nella fiducia delle tue sole *pupille*.